
TEAM UW SYFI: AGENT-ASSISTED TRACK WRITE-UP

FLASHINFER AI KERNEL GENERATION CONTEST @ MLSys 2026

Keisuke Kamahori¹ Steven Gao¹ Shihang Li¹ Wei Shen¹ Yile Gu¹

ABSTRACT

We describe Team UW SyFI’s submission to the agent-assisted track of the FlashInfer AI Kernel Generation Contest at MLSys 2026, in which we placed second in the GDN track. Our workflow is a tight human-in-the-loop collaboration in which the human provides high-level direction and a coding agent (Claude Code with Claude Opus 4.7, or Codex CLI with GPT 5.5) authors every line of code. The submission covers all five contest definitions: four are bf16 or FP8 CUDA kernels targeting sm_100, and one is a deliberately torch-only Python kernel chosen for bit-exact accumulation order. We summarize each kernel’s design and the collaboration pattern that produced it.

1 SOLUTION OVERVIEW

Team UW SyFI placed second in the GDN track of the agent-assisted contest. The submission repository is at <https://github.com/kamahori/mlsys-contest-syfi-agent-assisted>.

We adopted a tight human-in-the-loop workflow. At each step the human provided high-level direction (which language to start in, when to port to CUDA, which tensor-core API to apply, which Nsight Compute counter to inspect next), while the coding agent authored every line of source. The agent ran as either Claude Code with Claude Opus 4.7 or Codex CLI with GPT 5.5, and we switched between the two interchangeably. The submission covers all five contest definitions and ships the kernels listed in Table 1. All four CUDA kernels bind through tvn-ffi in destination-passing style. The one Python entry, the indexer, is a deliberate exception that we discuss in Section 2.4.

2 KERNEL DESIGN AND OPTIMIZATIONS

2.1 moe: FP8 block-scale fused MoE

The kernel is a single CUDA translation unit that fuses an eleven-stage pipeline. The stages, in order, are routing, permutation counting, prefix-sum offset computation, permutation fill, hidden-state dequantization, gather, weight dequantization, the bf16 GEMM, SwiGLU, final accumulation, and a final cast. Routing follows the

¹University of Washington, Seattle, Washington, USA. Correspondence to: Keisuke Kamahori <kamahori@cs.washington.edu>.

DeepSeek-V3 no-aux variant of group top- k , with an argmax-with-lane-tiebreak so that group scores remain deterministic under ties. Both GEMMs run on bf16 `nvcuda::wmma` tensor cores after FP8 block-scale dequantization, and the final scatter accumulates in fp32 via `atomicAdd` before casting to bf16 once per token. The contest-fixed constants ($H=7168$, $I=2048$, $BS=128$, $E_LOCAL=32$, $E_GLOBAL=256$, $TOP_K=8$, $N_GROUP=8$, $TOPK_GROUP=4$) are baked in as `constexpr`, allowing the compiler to unroll loops and fix tile sizes throughout.

2.2 gdn_prefill: chunked WY-form delta rule

This bf16 tensor-core kernel processes chunks of $C = 16$ tokens. The gated delta rule is rewritten into the un-gated WY form

$$\hat{s}_t = \hat{s}_0 + \sum_{s \leq t} \beta_s k_s u_s^T,$$

where u is the residual of the triangular solve $(I + L)u = \tilde{v} - K\hat{s}_0$. Each thread block uses four warps (128 threads) and assigns the four intra-chunk matmuls (KS, QS, KK, QK) one per warp. Attention and the $K^T \cdot U$ state update are split by output tile across warps. The Q, K, and V loads use double-buffered `cp.async.cg` to overlap with compute. We pad the shared-memory row stride to $K_PAD = K_DIM + 16 = 144$ bf16 elements, which breaks the 128-stride bank conflict that otherwise serializes `ldmatrix`.

2.3 gdn_decode: single-warp register-tiled state

The grid has $B \cdot H_v \cdot V/V_{TILE} = B \cdot 8 \cdot 16$ blocks, each consisting of a single warp. Each warp owns 8 v-rows, and each thread holds $K_LOCAL = K/32 = 4$ K-slots, so the per-warp state is 32 fp32 values held in registers. K-reductions are warp shuffles, and the old/new state and

Table 1. Submitted kernels (agent-assisted track).

Definition	Lang	File
moe_fp8_block_scale_ds_routing_topk8_ng8_kg4_e32_h7168_i2048	CUDA	moe/solution/cuda/moe.cu
gdn_prefill_qk4_v8_d128_k_last	CUDA	gdn_prefill/solution/cuda/gdn_prefill.cu
gdn_decode_qk4_v8_d128_k_last	CUDA	gdn_decode/solution/cuda/gdn_decode.cu
dsa_topk_indexer_fp8_h64_d128_topk2048_ps64	Python	dsa_indexer/solution/python/dsa_topk_indexer.py
dsa_sparse_attention_h16_ckv512_kpe64_topk2048_ps64	CUDA	dsa_sparse_attention/solution/cuda/dsa_sparse_attention.cu

inputs are loaded with `float4` vectorization. The output is derived from `qS(oid)`, `kS(oid)`, and the `qk` dot product, which avoids a second pass over the updated state.

2.4 dsa_topk_indexer: deliberately torch-only

This kernel is implemented in pure PyTorch rather than Triton or CUDA. A Triton fused-scorer version produces the correct top- K *set* but a different ordering, because fp32 accumulation order differs from torch’s native `matmul`. The contest correctness check is position-wise, so a permuted ordering registers as a large absolute error. Using torch `matmul` guarantees identical accumulation order, identical scores, identical `torch.topk` output, and therefore correctness.

We add two optimizations beyond plain torch. First, an in-process LRU cache keyed by the tuple of data pointer, shape, stride, dtype, and device avoids recomputation when repeated calls receive the same input tensors. Second, a custom FP8 KV-cache dequantization path decodes the contest’s per-page layout, which is 64×128 fp8 values followed by 64 fp32 scales, with stride `HEAD_DIM_SF = 132`.

2.5 dsa_sparse_attention: flash-decoding with WMMA

Each CTA holds one tile and runs 256 threads (eight warps). It reads `BLOCK_N` KV tokens, computes $[H=16, D_{TOT}=576] \times [D_{TOT}, BLOCK_N]^T$ on the WMMA tensor cores, applies softmax one row at a time, and then computes $[H, BLOCK_N] \times [BLOCK_N, D_C=512]$ for the PV multiplication. The split count `K_SPLITS` is dispatched on the host based on `Nt`: a small `Nt` receives more splits in order to fill the machine, while a large `Nt` receives fewer splits so that each CTA carries more work and partial-O traffic stays bounded. We pad the shared-memory stride to `D_TOT_PAD = D_TOT + 8 = 584`, shifting the per-row bank offset by four to break the $D_{TOT} \times 2B = 1152B$ periodicity that would otherwise serialize `ldmatrix`.

3 TOOLS AND LANGUAGES

The four CUDA kernels target `sm_100` (Blackwell), use bf16 storage with FP8 inputs, and bind through `tvm-ffi` in destination-passing style; the single Python entry uses torch. On the GPU side we use `nvcuda::wmma` for the bf16 matrix multiplications, `cp.async.cg` for shared-memory staging, and `ldmatrix` for the WMMA fragment loads. The coding agent runs as either Claude Code with Claude Opus 4.7 or Codex CLI with GPT 5.5, and we use the two interchangeably for code authoring, refactoring, and Nsight Compute driven retuning. We use Nsight Compute for speed-of-light, roofline, and bank-conflict analysis. Correctness and performance evaluation is provided by `flashinfer-bench`, with `pack_solution.py`, `run_local.py`, and `run_modal.py` entries per kernel directory; we use Modal B200 for benchmarking when local GPUs are contended.

4 HUMAN AND AGENT CONTRIBUTIONS

The human did not write or edit any kernel source. Every line of Triton, CUDA, and Python in the submission was authored by the agent. The collaboration pattern was the same on every kernel. We first asked the agent to write a working Triton implementation against the PyTorch reference, since Triton is the path of least resistance for getting to correctness and iterating quickly. Once the Triton version passed, we asked the agent to translate the kernel to CUDA. We then ran Nsight Compute and iterated, with the human supplying high-level direction such as “use tensor cores here”, “try warp specialization”, “look at the shared-memory bank-conflict counter”, or “the min-speedup workload is at small `Nt`, so make the schedule shape-adaptive”. The agent performed the diagnosis, the editing, and the reprofiling, while the human chose the next thing to chase.